

Motivation & Problem Statement :

According to the International Diabetes Federation, the occurrence of diabetes in the adult population (20-79 years) is projected to increase by **46% by the year 2045**, with South Asians being the most susceptible to this issue. This is because **South Asians are predisposed** to the development of diabetes due to biological and lifestyle factors ([Source 1](#)). Diabetic retinopathy is a consequential complication of diabetes, characterized by damage to the retina at the back of the eye. By utilizing technologies such as machine learning and computer vision algorithms, we aim to create a robust model capable of **accurately detecting** and assessing diabetic retinopathy from retina images. This solution would not only streamline the diagnostic process but also offer the potential for **early detection**, enabling timely interventions and treatment plans. This growing issue can thus be handled effectively by the healthcare industry.

Dataset & Data Source :

The dataset comprises of fundus images in IDRiD captured by a retinal specialist at an Eye Clinic located in Nanded, Maharashtra, India, with reasonable mixture of disease stratification representative of **diabetic retinopathy (DR)** and **diabetic macular edema (DME)** is present. The data also provides information regarding disease severity level of DR and DME. The grading for all images is available in the CSV file categorized according to the International Clinical Diabetic Retinopathy Scale: 0 - No abnormalities, 1 - Mild nonproliferative diabetic retinopathy, 2 - Moderate nonproliferative diabetic retinopathy, 3 - Severe nonproliferative diabetic retinopathy, 4 - Proliferative diabetic retinopathy.

There are **516 images** that have a resolution of **4288×2848 pixels** and are stored in jpg file format. The size of each image is about 800 KB.

Data Source : [Data Source](#)

Data Download Link : [Download Link](#)

Proposed Methodologies :

Given the problem at hand, we will first **reduce the size of the images** by decreasing the resolution to make them more compatible for a small neural network, as 4288×2848 would be too large of an input size for us. After reducing the image resolution, we will then apply other data preprocessing techniques such as cropping, resizing, filtering, or enhancing as required. To clean the data, we could also shear, **flip**, **rotate**, **scale**, translate, adjust brightness, adjust contrast, inject noise, or blur the image. Lastly, we can also perform more advanced preprocessing techniques, such as grayscale conversion, blackhat filtering, and thresholding for inpainting, to remove noise from the images, as shown in this paper ([Source 2](#)). Now that we have finished preprocessing, we can create a model. As stated in the guidelines, we will initially use a simple non-deep learning method; in this scenario, we will use **logistic regression**. Afterwards, we will explore deep learning methods, such as a **simple MLP** (Multi-Layer Perceptron), with and without linearity, and progress to more complex models, such as **Convolutional Neural Networks (CNNs)**. We would use different types of optimizers, such as **SGD**, **Adagrad**, or more sophisticated methods like **Adam**, which hopefully will improve the model's performance. Given the multiclass nature of the data, we would use **cross-entropy** as the metric for evaluation. If the performance of our model is not up to standard, we even might consider training on a pretrained model.

Project Execution :

Data Augmentation and Data Loading:

Initially, we organized the images by utilizing a CSV file containing the actual labels, **streamlining** the folder organization process. Experimenting with various data augmentation techniques, we **referenced industry-standard** healthcare data processing papers to refine our choices. Despite this, lacking domain expertise led to uncertainty in selecting transformations, necessitating extensive iterations through experimentation. Ultimately, we settled on **random rotation, random flip**, and **resizing** the images to fit within specified dimensions, with **padding** to preserve information. We then **normalized** the training and validation data based on channel means and standard deviations from the training data. Using DataLoader, we loaded images for training and testing, applying augmentation, resizing, padding, and tensor transformation. Testing data underwent only resizing, padding, and tensor transformation without augmentation. We then split the training data into 80% for training and 20% for validation.

Modeling:

We decided to **experiment** with different models, since this would allow us to find different ways to approach the problem and to see if the problem is a simple problem or a complex problem that requires the use of neural networks. We first used:

Non Deep Models - Logistic regression and K-Means:

We started off by using supervised and unsupervised machine learning classification models - **Multiclass Logistic Regression** and **K-Means** to get an idea of how the dataset performs. We flattened the images based on the pixel data and passed it through the regression model with 5 classes, which gave us an accuracy of 36%. The binary K-Means classification (no abnormalities and have DR) gave us an accuracy of 51% which is similar to a random guess. The 5 cluster classification gave a bit better results than random guess with a test accuracy of 28%.

Convolutional Neural Network (CNN):

We built a very simple CNN with **4/3 convolutional layers**, utilizing **ReLU** non-linearity and **max-pooling**. The purpose was to capture patterns and reduce the dimensionality of the data for classification. We then flattened the results and passed them into a linear layer for classification into 5 categories. However, this simple model struggled to capture patterns in the images and tended to predict everything as the majority class, resulting in either 0 or 2 predictions. We attempted to perform some **hyperparameter tuning**, but we noticed that the model's performance remained subpar, likely due to its relatively small size compared to industry standards. This led us to employ transfer learning and pre-built models.

Vision Transformer (ViT) :

We began with ViT-16, partitioning data into 16 patches and processing images at [224,224]. To manage its size, we modified the final layer into a small MLP with one hidden layer and one output layer for classification, resulting in **77,000** tunable parameters out of **85** million. While outperforming simpler CNNs, ViT encountered challenges, often misclassifying most training data as 0, with some as 2 or 4, indicating ongoing struggles with generalization and feature recognition.

EfficientNet :

Given that the previous model did not perform as well as expected, we opted to use a smaller model but **fine-tuned all the parameters**. This model had approximately **8 million parameters**. We modified the last layer into a linear layer for classification and fine-tuned the model for 10 epochs. This resulted in a better model than before, capable of capturing more of the variance in the data. Indeed, the model now **predicts all 5 labels** and generalizes better, indicating improvement in our classification task.

ResNet:

We employed the ResNet18 model, pretrained on large-scale images, featuring 18 layers with convolutional, pooling, fully connected layers, and **residual connections**. We adapted the final layer to our data while keeping initial layers **unfrozen** for fine-tuning during training. ResNet's sophistication enabled it to capture intricate data patterns, yielding significant improvements over baseline CNN, achieving a 70% validation accuracy and effectively handling class imbalances.

Findings & Complications:

Despite sufficient computational resources and improvements in training and validation, testing accuracy did not meet expectations. Potential factors include **overfitting**, **dataset shift**, and issues with **model generalization**, exacerbated by data augmentation only applied to training and validation. Experimentation without normalization and augmentation yielded **similar results** across datasets, indicating other contributing factors. The top-performing model in terms of training and validation accuracy was EfficientNet, achieving nearly **95% and 90%**, respectively, with a learning rate of 0.0001 and a batch size of 64. ResNet attained approximately **40%** training accuracy, emerging as the best-performing model among those tested.

Future Steps

Moving forward, we can try to get **more data** for model training which can potentially help in improving the accuracy and help in better generalization of the model. Since the classes of the data have an **imbalance**, we can also apply techniques to the data to **upsample the minority** classes so that the model is able to capture variation with respect to the risk stage of this condition. Lastly, we can also experiment further with **data augmentation** and **image transformations**, given more access to sophisticated resources and no time bound constraints. All this will aid in the better performance of the model.

Conclusion

In these times of the rapidly growing rate of diabetes, our model holds a high importance with a wide array of practical business implications.

- The model can be utilized in healthcare to streamline the diagnostic process for diabetic retinopathy and offer the potential for **early detection** of this very major issue.
- Moreover, it can also be **extrapolated and fine-tuned** to be applied for other medical imaging modalities for the purpose of diagnostic processes, especially for medical imaging in the realm of optics.
- Furthermore, it can also be utilized by organizations in the intersection of healthcare and technology for **remote diagnosis** of diabetic retinopathy. Patients can perhaps obtain images of the retina through a fundus camera, which can further be fed into the model for remote diagnosis.

Reports:

Appendix:

[CNN Hyper Tuning 3 CNN](#)

[CNN Hyper Tuning 4 CNN](#)

[ViT](#)

[EfficientNet](#)

[ResNet](#)

Dataset Distribution:

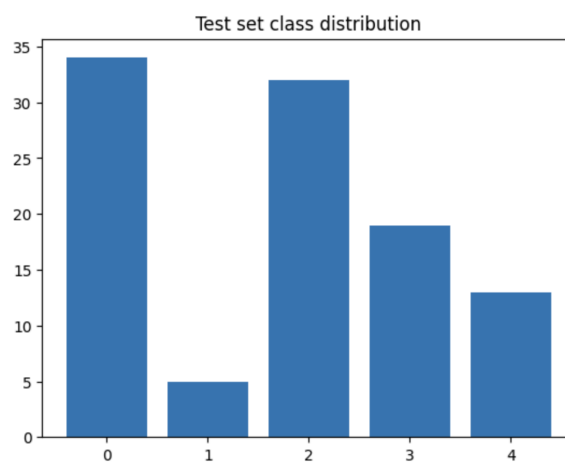
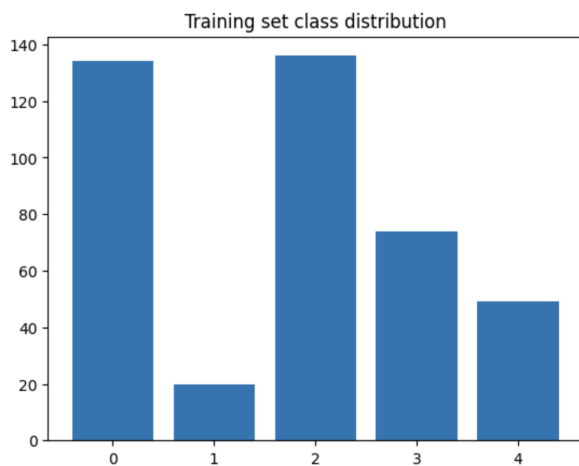
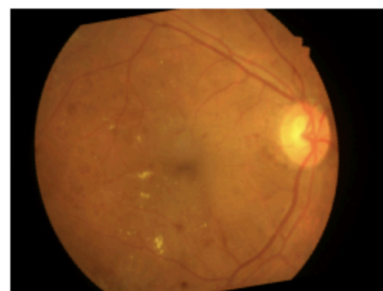
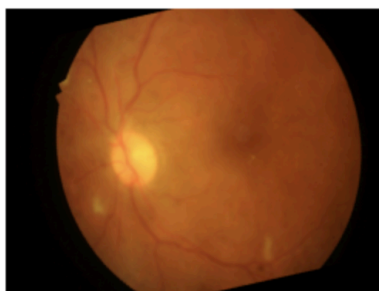
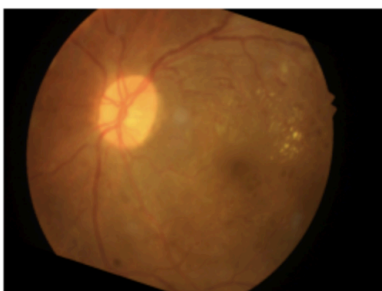
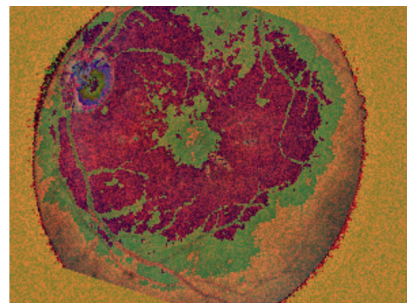
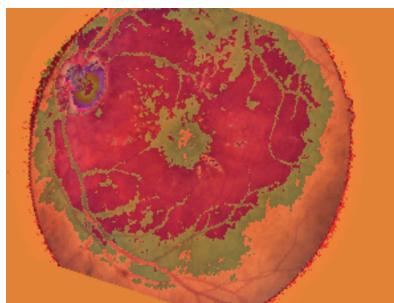
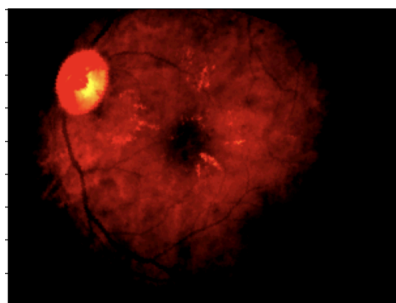


Image Transformations:

Horizontal flips and Rotation:



Color Transformations:



KMeans Binary:

Test Accuracy = 52%

	precision	recall	f1-score	support
0	0.34	0.47	0.40	34
1	0.68	0.55	0.61	69
accuracy			0.52	103
macro avg	0.51	0.51	0.50	103
weighted avg	0.57	0.52	0.54	103

KMeans 5 Clusters:

Test Accuracy = 28%

	precision	recall	f1-score	support
0	0.50	0.26	0.35	34
1	0.25	0.40	0.31	5
2	0.27	0.25	0.26	32
3	0.11	0.05	0.07	19
4	0.24	0.69	0.35	13
accuracy			0.28	103
macro avg	0.27	0.33	0.27	103
weighted avg	0.31	0.28	0.27	103

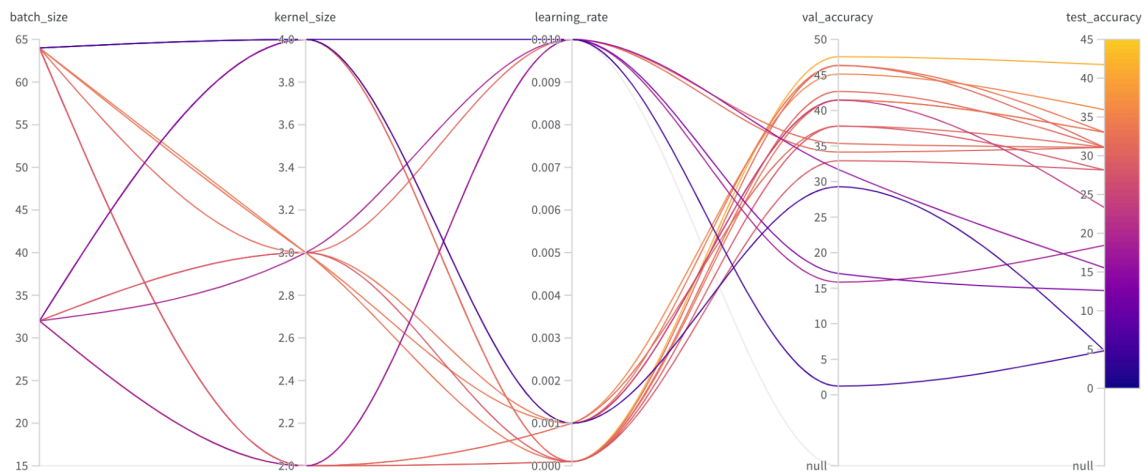
CNN Model Architecture:

Layer (type:depth-idx)	Output Shape	Param #
CNN	[32, 5]	--
└Sequential: 1-1	[32, 5]	--
└Conv2d: 2-1	[4, 446, 498]	112
└ReLU: 2-2	[4, 446, 498]	--
└Conv2d: 2-3	[8, 444, 496]	296
└ReLU: 2-4	[8, 444, 496]	--
└Conv2d: 2-5	[16, 442, 494]	1,168
└ReLU: 2-6	[16, 442, 494]	--
└MaxPool2d: 2-7	[16, 221, 247]	--
└Conv2d: 2-8	[32, 219, 245]	4,640
└MaxPool2d: 2-9	[32, 109, 122]	--
└Flatten: 2-10	[32, 13298]	--
└Linear: 2-11	[32, 5]	66,495
Total params: 72,711		
Trainable params: 72,711		
Non-trainable params: 0		
Total mult-adds (M): 49.13		
Input size (MB): 2.69		
Forward/backward pass size (MB): 62.89		
Params size (MB): 0.29		
Estimated Total Size (MB): 65.87		

WandB Report for hyperparameter Tuning:

Published Report Link: <https://api.wandb.ai/links/pmpbu/o17j4j23>

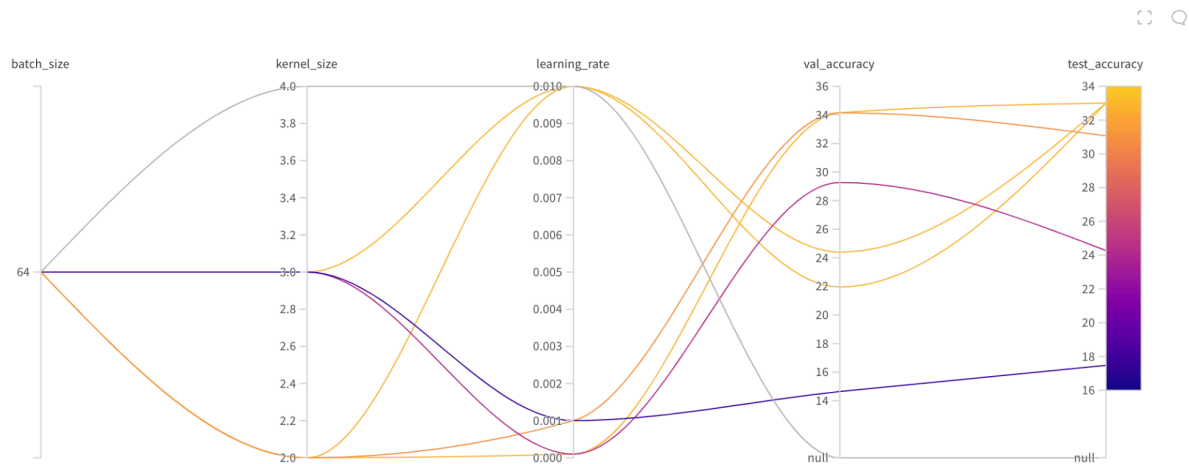
Another Report Link: [CNN](#)



Best model parameters in the sweeps:

- Batch = 32
- Learning Rate = 0.0001
- Kernel = 2
- Validation Accuracy = 47.56
- Test Accuracy = 41.74

Class Imbalance:

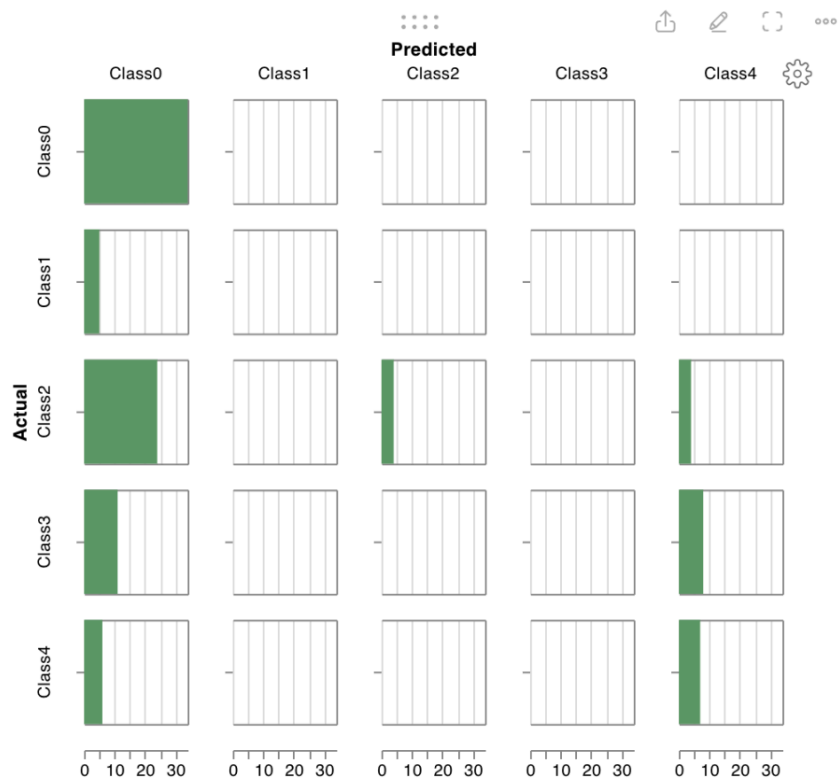


Report: [Class Imbalance](#)

Vision Transformer:

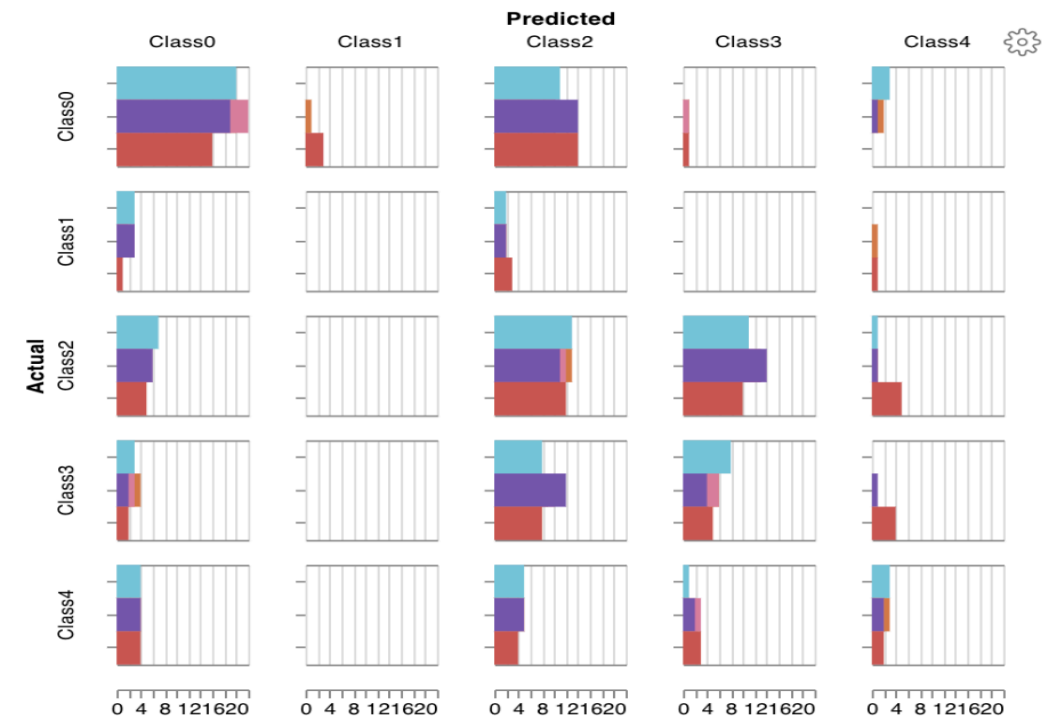
Report Link: [ViT](#)

Layer (type:depth-idx)	Param #
VisionTransformer	768
└Conv2d: 1-1	(590,592)
└Encoder: 1-2	151,296
└Dropout: 2-1	--
└Sequential: 2-2	--
└EncoderBlock: 3-1	(7,087,872)
└EncoderBlock: 3-2	(7,087,872)
└EncoderBlock: 3-3	(7,087,872)
└EncoderBlock: 3-4	(7,087,872)
└EncoderBlock: 3-5	(7,087,872)
└EncoderBlock: 3-6	(7,087,872)
└EncoderBlock: 3-7	(7,087,872)
└EncoderBlock: 3-8	(7,087,872)
└EncoderBlock: 3-9	(7,087,872)
└EncoderBlock: 3-10	(7,087,872)
└EncoderBlock: 3-11	(7,087,872)
└EncoderBlock: 3-12	(7,087,872)
└LayerNorm: 2-3	(1,536)
└Sequential: 1-3	--
└Linear: 2-4	76,900
└ReLU: 2-5	--
└Linear: 2-6	505
Total params: 85,876,061	
Trainable params: 77,405	
Non-trainable params: 85,798,656	



EfficientNet:
 Report: [EfficientNet](#)

EfficientNet	---
Sequential: 1-1	---
Conv2dNormActivation: 2-1	---
Conv2d: 3-1	864
BatchNorm2d: 3-2	64
SiLU: 3-3	---
Sequential: 2-2	---
MBConv: 3-4	1,448
MBConv: 3-5	612
Sequential: 2-3	---
MBConv: 3-6	6,004
MBConv: 3-7	10,710
MBConv: 3-8	10,710
Sequential: 2-4	---
MBConv: 3-9	16,518
MBConv: 3-10	43,308
MBConv: 3-11	43,308
Sequential: 2-5	---
MBConv: 3-12	50,300
MBConv: 3-13	123,750
MBConv: 3-14	123,750
MBConv: 3-15	123,750
Sequential: 2-6	---
MBConv: 3-16	149,158
MBConv: 3-17	237,870
MBConv: 3-18	237,870
MBConv: 3-19	237,870
Sequential: 2-7	---
MBConv: 3-20	301,406
MBConv: 3-21	686,868
MBConv: 3-22	686,868
MBConv: 3-23	686,868
MBConv: 3-24	686,868
Sequential: 2-8	---
MBConv: 3-25	846,900
MBConv: 3-26	1,888,920
Conv2dNormActivation: 2-9	---
Conv2d: 3-27	495,616
BatchNorm2d: 3-28	2,816
SiLU: 3-29	---
AdaptiveAvgPool2d: 1-2	---
Sequential: 1-3	---
Dropout: 2-10	---
Linear: 2-11	140,900
ReLU: 2-12	---
Linear: 2-13	505
=====	
Total params: 7,842,399	
Trainable params: 7,842,399	
Non-trainable params: 0	
=====	



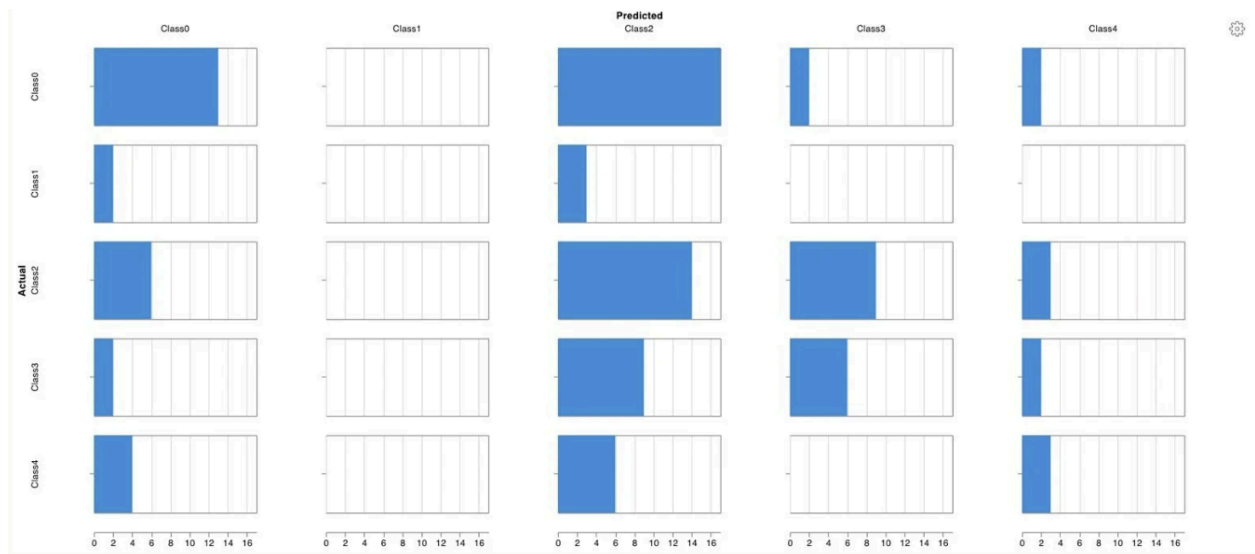
ResNet:

Link: [ResNet](#)

Last layer that we changed:

```
└─AdaptiveAvgPool2d: 1-9      --
└─Sequential: 1-10           --
    └─Linear: 2-9             51,300
    └─ReLU: 2-10             --
    └─Linear: 2-11            505
```

```
=====
Total params: 11,228,317
Trainable params: 11,228,317
Non-trainable params: 0
=====
```



Github Link: [GitHub](#)

Notion Link: [Contribution Table](#)